Kolektor zdarzeń Customer 360

Propozycja prac Development dla Grupy ORLEN: kod event-collector-api jako niezależny projekt

ORLEN · wersja 1.0 · 5 października 2026 r.

Czas do gotowości produkcyjnej kolektora	Serwis
13 tygodni od startu prac, razem z infrastrukturą z propozycji DevOps; podłączanie źródeł według gotowości spółek	1: event-collector-api na Cloud Run

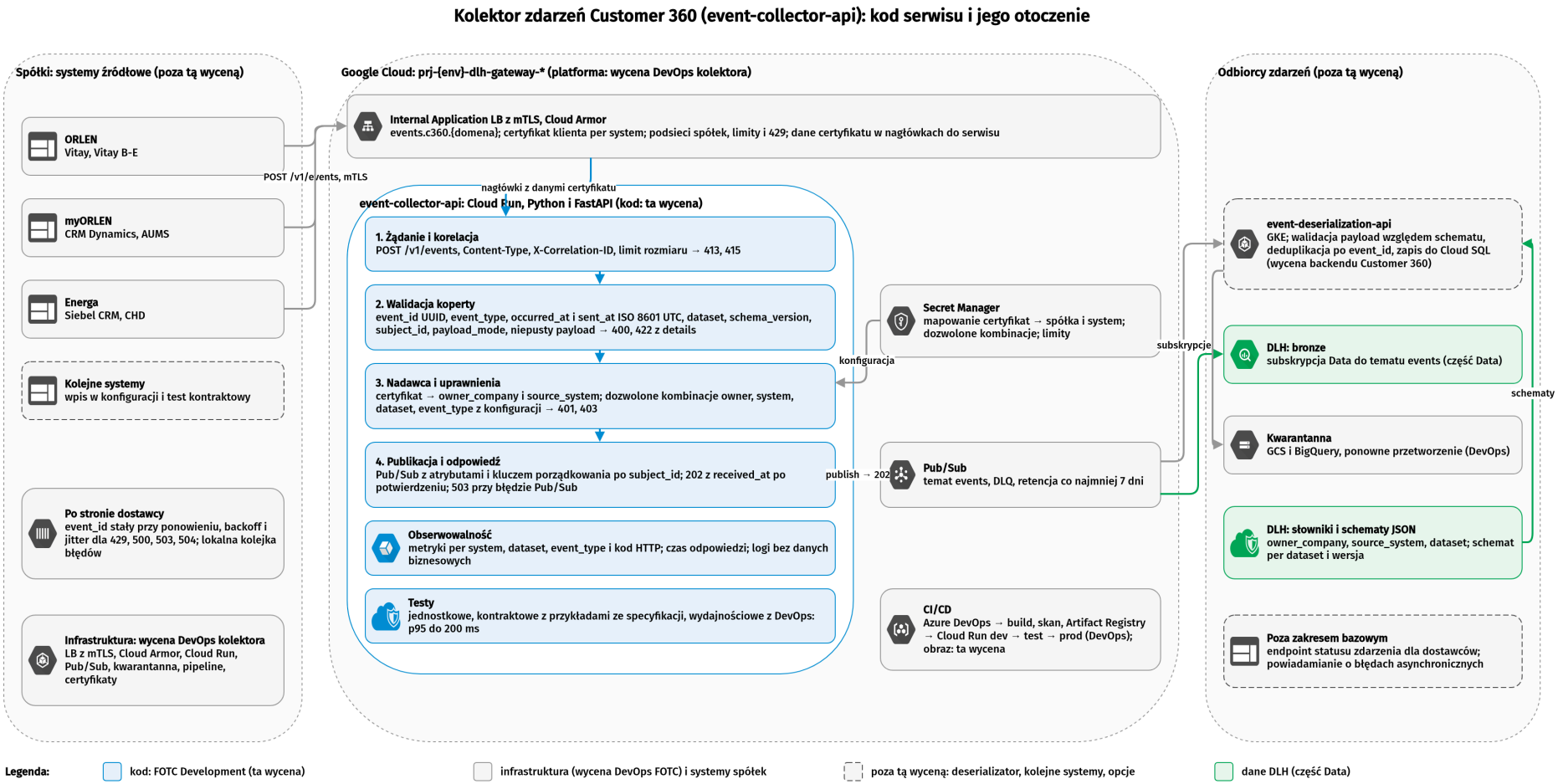
Czego potrzebuje ORLEN

Ten dokument jest odpowiedzią na specyfikację API kolektora zdarzeń (event-collector-api v0.1). Kolektor to jedno wejście dla zdarzeń o klientach z systemów ORLEN, myORLEN i Energa, dzięki którym konsultant wspólnego contact center widzi w Widoku 360 zmiany z tego samego dnia: system źródłowy wysyła zdarzenie przez POST /v1/events po mTLS, kolektor sprawdza kopertę i uprawnienia nadawcy, odpowiada 202 Accepted po trwałym przyjęciu i publikuje zdarzenie do Pub/Sub. Traktujemy kolektor jako niezależny projekt. Ta propozycja obejmuje kod serwisu, jego testy i uruchomienie ze spółkami. Infrastruktura kolektora (load balancer z mTLS, Cloud Run, Pub/Sub, kwarantanna, pipeline) jest w propozycji DevOps kolektora, a walidacja payload względem schematów, deduplikacja i zapis do Cloud SQL są po stronie deserializatora w propozycji backendu Customer 360. Ta wycena nie sumuje się z żadną z nich.

Założenia

- Stack: Python 3.14 i FastAPI, zgodnie z sugestią. Obraz kontenera kolektora dostarcza zespół Development FOTC; Cloud Run, pipeline wdrożeniowy, Pub/Sub i kwarantannę dostarcza DevOps FOTC według propozycji kolektora.
- Zakres kodu to synchroniczna część specyfikacji: przyjęcie żądania, walidacja koperty i metadanych, identyfikacja i uprawnienia nadawcy, limity, odpowiedzi według katalogu, publikacja do Pub/Sub. Walidacja asynchroniczna payload, deduplikacja po event_id, kwarantanna i zapis danych są poza tą wyceną.
- Load balancer kończy mTLS i przekazuje dane certyfikatu klienckiego w nagłówkach; kolektor mapuje certyfikat na owner_company i source_system z konfiguracji w Secret Manager i dopuszcza tylko uzgodnione kombinacje owner_company, source_system, dataset i event_type.
- Parametry oznaczone w specyfikacji jako do ustalenia (limit rozmiaru, limity ruchu, payload_mode jako pole albo konfiguracja datasetu) rozstrzygamy w projekcie technicznym; w kodzie są parametrami konfiguracji.
- Wycena obejmuje sześć systemów źródłowych w trzech spółkach, jak w propozycji DevOps kolektora; słowniki i schematy JSON zdarzeń dostarcza zespół Data (DLH).
- Zespół: jeden developer. Czas oczekiwania na decyzje, dane i testy po stronie ORLEN i spółek nie wchodzi w harmonogram.

Rozwiązanie: ścieżka zdarzenia przez kolektor



Kolektor zdarzeń Customer 360: kod i otoczenie

Diagram pokazuje jedno środowisko; dev, test i prod mają ten sam układ. Wersja w pełnej rozdzielczości jest w osobnym pliku.

Rezultat dla ORLEN

- Kolektor przyjmuje zdarzenia zgodne ze specyfikacją v0.1 i odpowiada 202 dopiero po potwierdzeniu publikacji do Pub/Sub, więc 202 oznacza trwałe przyjęcie.
- Żądanie bez uprawnień do wskazanej kombinacji owner_company, source_system, dataset i event_type dostaje 403; niepoprawna koperta dostaje 400 albo 422 z listą pól i przyczyn.
- Dostawcy systemów źródłowych dostają kontrakt OpenAPI z przykładami i instrukcję z kodami błędów i polityką ponowień.
- Metryki żądań per system, dataset, typ zdarzenia i kod HTTP oraz czas odpowiedzi trafiają do monitoringu ORLEN.
- Nowy system źródłowy to wpis w konfiguracji i test kontraktowy, bez zmian w kodzie.

Decyzje projektowe i ich uzasadnienie

Decyzja	Dlaczego tak	Co to oznacza dla ORLEN
Kolektor waliduje tylko kopertę; payload jest walidowany asynchronicznie w deserializatorze	Specyfikacja (rozdział 9) rozdziela walidację na synchroniczną i asynchroniczną, żeby odpowiedź 202 mieściła się w 200 ms niezależnie od złożoności schematu.	Błąd w payload nie wraca w odpowiedzi HTTP, tylko trafia do kwarantanny; dostawcy potrzebują informacji o błędach asynchronicznych (pozycja poza zakresem bazowym).
202 dopiero po potwierdzeniu publikacji do Pub/Sub	Specyfikacja (rozdział 3): 202 oznacza trwałe przyjęcie. Odpowiedź przed potwierdzeniem mogłaby zgubić zdarzenie przy awarii.	Przy niedostępności Pub/Sub kolektor odpowiada 503, a dostawca ponawia z tym samym event_id.
Tożsamość nadawcy z nagłówków load balancera, nie z własnej obsługi mTLS	Load balancer kończy mTLS i weryfikuje certyfikat (propozycja DevOps); kolektor tylko mapuje dane certyfikatu na spółkę i system.	Jedno miejsce zarządzania certyfikatami; zmiana modelu LB wymaga zmiany kolektora.
Dozwolone kombinacje owner_company, source_system, dataset i event_type w konfiguracji, nie w kodzie	Rejestracja nowego typu zdarzenia (rozdział 6 specyfikacji) to decyzja DLH i wpis w konfiguracji, bez wydania serwisu.	Nowe źródło lub typ zdarzenia bez zmian w kodzie; treść rejestru po stronie Data.
Klucz porządkowania komunikatu po subject_id	Specyfikacja (rozdział 8.3) wymaga kolejności zdarzeń dotyczących tego samego obiektu, nie globalnej.	Odbiorcy dostają zdarzenia jednego klienta po kolei; różnych klientów równolegle.
Parametry oznaczone w specyfikacji jako do ustalenia są konfiguracją	Limit rozmiaru, limity ruchu i payload_mode zmienią się po testach wydajnościowych; zmiana nie może wymagać wydania.	Decyzje w fazie A, strojenie w fazie C bez zmian w kodzie.

FOTC

Zakres prac i harmonogram

Epik	Nazwa
K1	Projekt techniczny i kontrakt API
K2	Implementacja event-collector-api
K3	Testy kontraktowe i wydajnościowe
K4	Uruchomienie per źródło i przekazanie

FOTC odpowiada za kod kolektora, kontrakt OpenAPI, testy razem z DevOps, wsparcie podłączenia sześciu systemów źródłowych, wdrożenie produkcyjne i dwa tygodnie wsparcia po uruchomieniu. Największa pozycja to implementacja: walidacja koperty, identyfikacja nadawcy, publikacja i metryki. Testy jednostkowe są w każdym epiku. Przy wspólnym zamówieniu z backendem Customer 360 szablon serwisu i bibliotekę wspólną liczymy raz i aktualizujemy wycenę przed startem.

Poza zakresem bazowym

- **Endpoint statusu zdarzenia dla dostawców.** Deserializator musi utrwalać status każdego zdarzenia, a dostawca widzi tylko własne. GET /v1/events/{event_id} ze stanem przyjęcia i wynikiem walidacji asynchronicznej. Odpowiada na pytanie 13 specyfikacji.
- **Powiadamianie dostawców o błędach asynchronicznych: raport dzienny albo callback do systemów źródłowych.** Po odrzuceniu zdarzenia w walidacji asynchronicznej; callback wymaga ruchu wychodzącego po stronie DevOps. Odpowiada na pytanie 7 specyfikacji.
- **Kolejny system źródłowy.** Konfiguracja kombinacji i certyfikatu, test kontraktowy; prace sieciowe i certyfikaty w propozycji DevOps.

Harmonogram, ryzyka i kolejny krok

Harmonogram

Faza	Czas od startu	Zakres
A. Projekt i kontrakt	tygodnie 1–4	K1, szkielet serwisu; kontrakt OpenAPI przekazany spółkom razem z procedurą podłączenia DevOps
B. Implementacja na dev i test	tygodnie 5–9	K2 na dev i test, pierwsze testy łączności ze spółkami
C. Testy i gotowość produkcyjna	tygodnie 10–13	K3 na środowisku test razem z testami wydajnościowymi DevOps; kolektor gotowy produkcyjnie
D. Podłączanie źródeł	od tygodnia 14 według gotowości spółek, plus 2 tygodnie wsparcia po ostatnim	K4: testy per system, wdrożenie produkcyjne, wsparcie po uruchomieniu, dokumentacja, przekazanie

Ryzyka

Ryzyko	Postępowanie
Parametry specyfikacji do ustalenia: limit rozmiaru, limity ruchu, payload_mode	Decyzje w K1 z DLH i DevOps, wszystkie jako konfiguracja
Sposób przekazania tożsamości certyfikatu z load balancera do serwisu	Potwierdzenie w K1 z DevOps, test obcego certyfikatu w K3
Gotowość systemów źródłowych do testów w trzech spółkach	Testy kontraktowe na przykładach ze specyfikacji; podłączenie według gotowości
Partie zdarzeń (do 1000 rekordów) a limit rozmiaru	Limit i podział partii uzgodnione w K1, test w K3
Żaden dostawca nie podał wolumenów ani listy zdarzeń; na dziś uzgodniony jest jeden scenariusz (rejestracja klienta)	Formularz z rozdziału 17 specyfikacji per system w fazie A; testy wydajnościowe K3.2 wymagają wolumenów przed fazą C

Dziękujemy za możliwość przedstawienia tej propozycji.